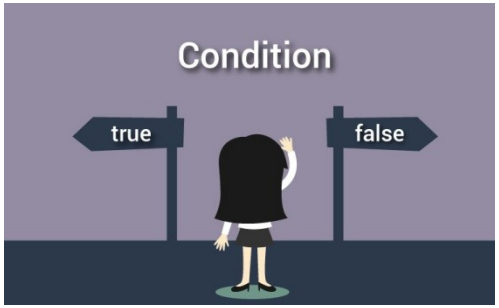
Looping

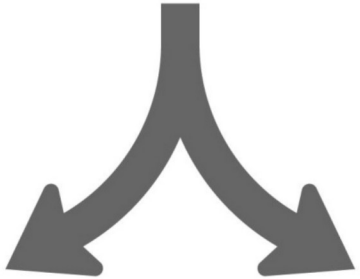
CSL100

August 22, 2024

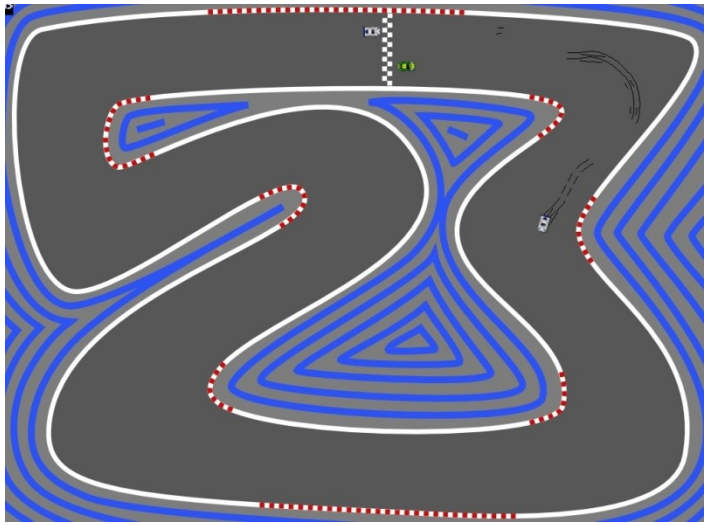
Control Statements

- Branching





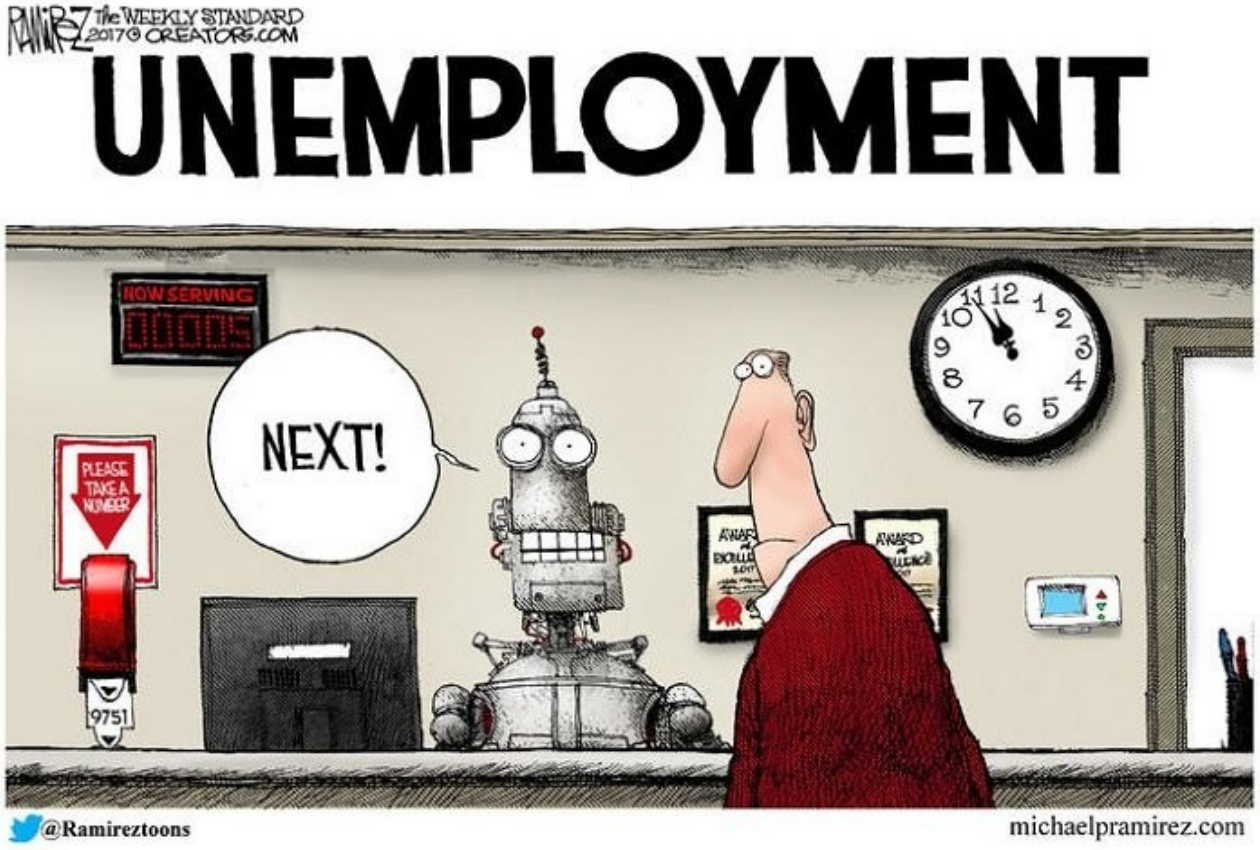
- Looping



This Class

- Introduction to looping
- While statement
- Do-while statement
- For statement

The Computer's BIG Advantage



Once you've told a computer how to do something once, it can reproduce the exact same process a trillion times.

ATM

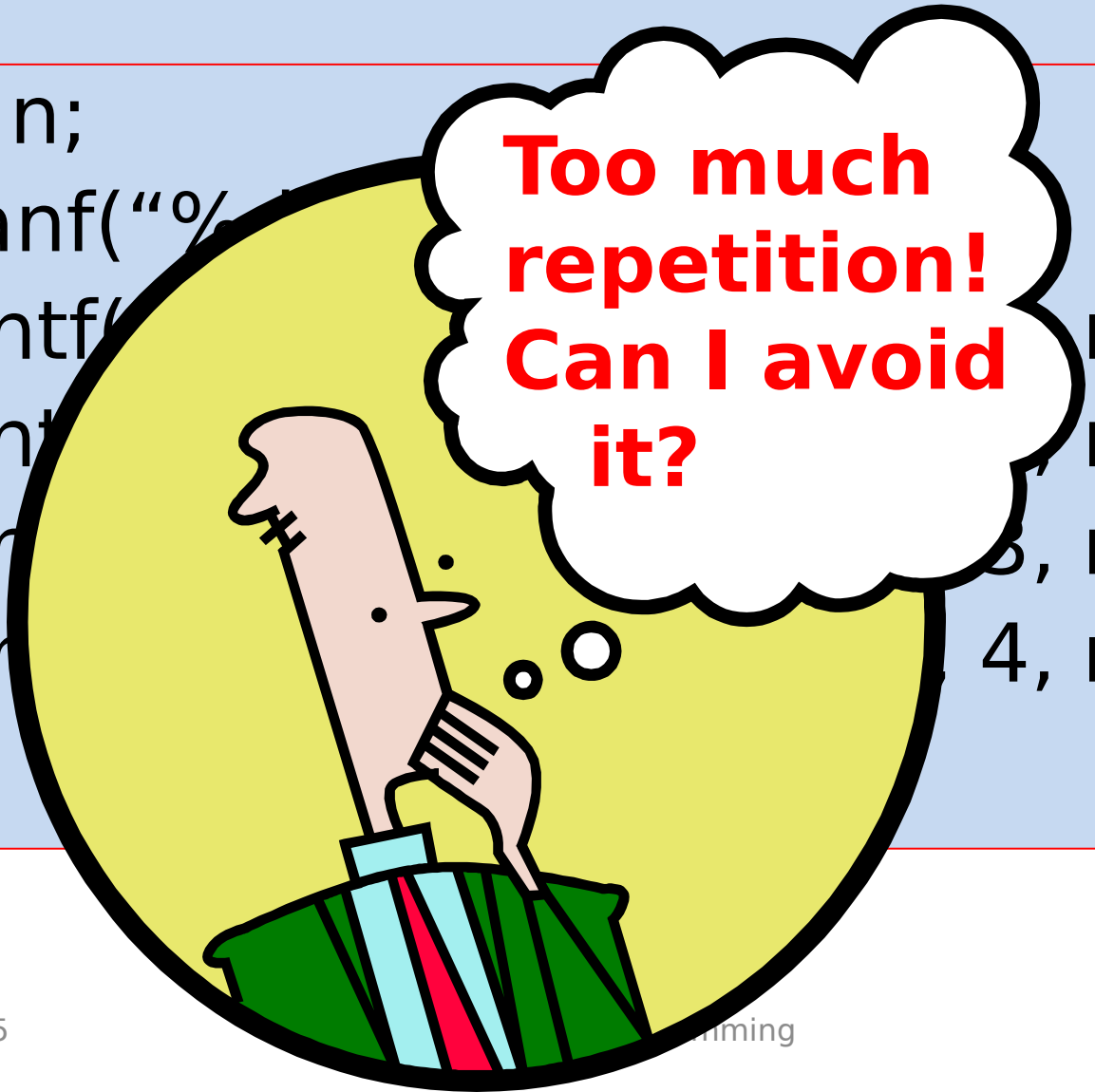


Printing Multiplication Table

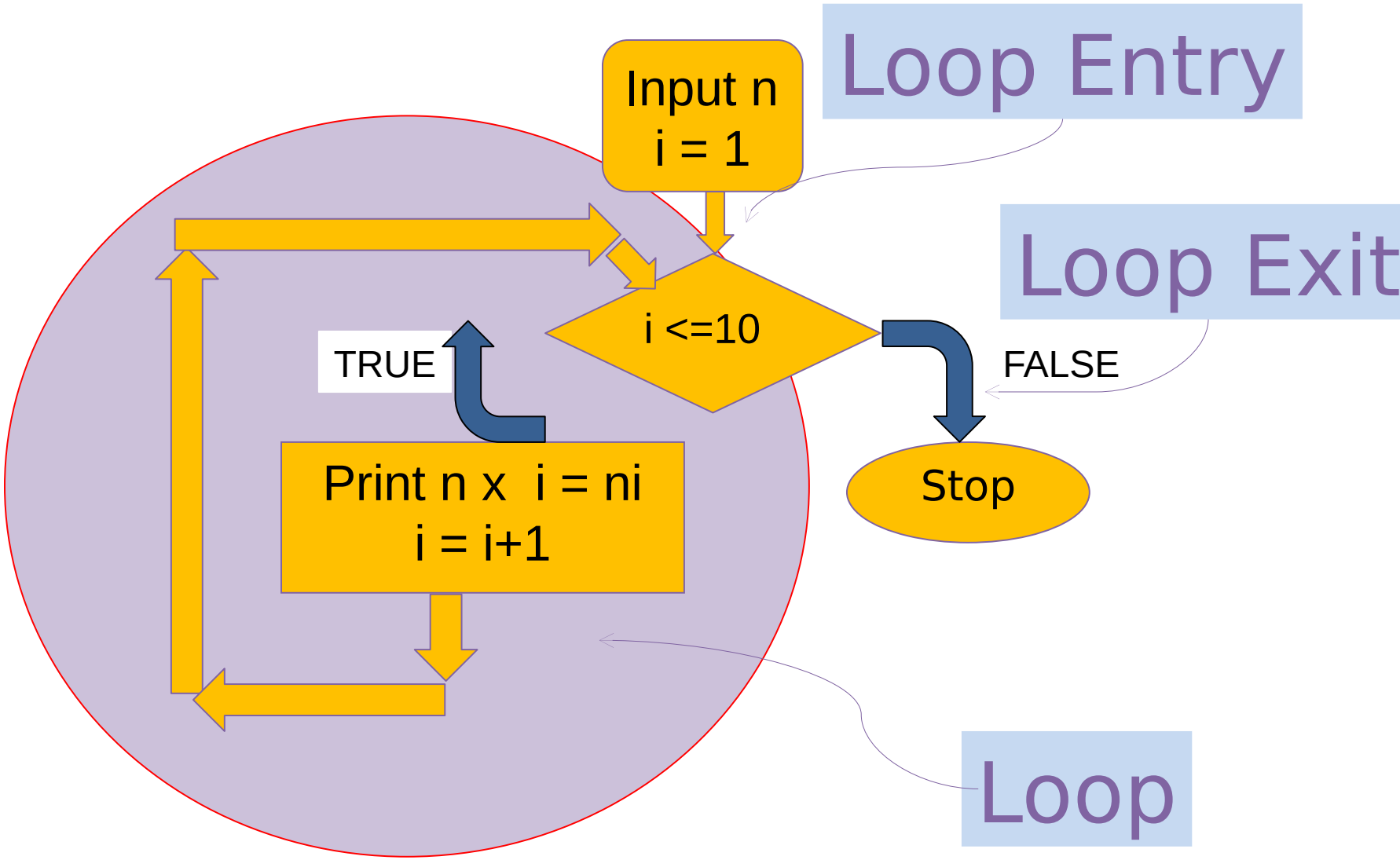
5	X	1	=	5
5	X	2	=	10
5	X	3	=	15
5	X	4	=	20
5	X	5	=	25
5	X	6	=	30
5	X	7	=	35
5	X	8	=	40
5	X	9	=	45
5	X	10	=	50

Program…

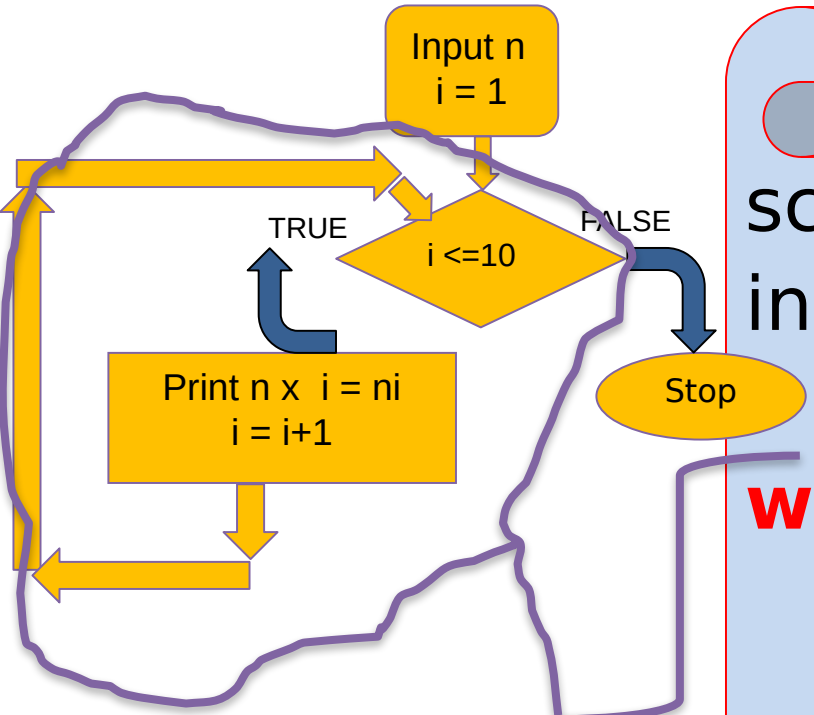
```
int n;
scanf(“%
printf(                                  n*1);
print                                    n*2);
prin                                     , n*3);
prin                                     4, n*4);
....
```



Printing Multiplication Table



Printing Multiplication Table



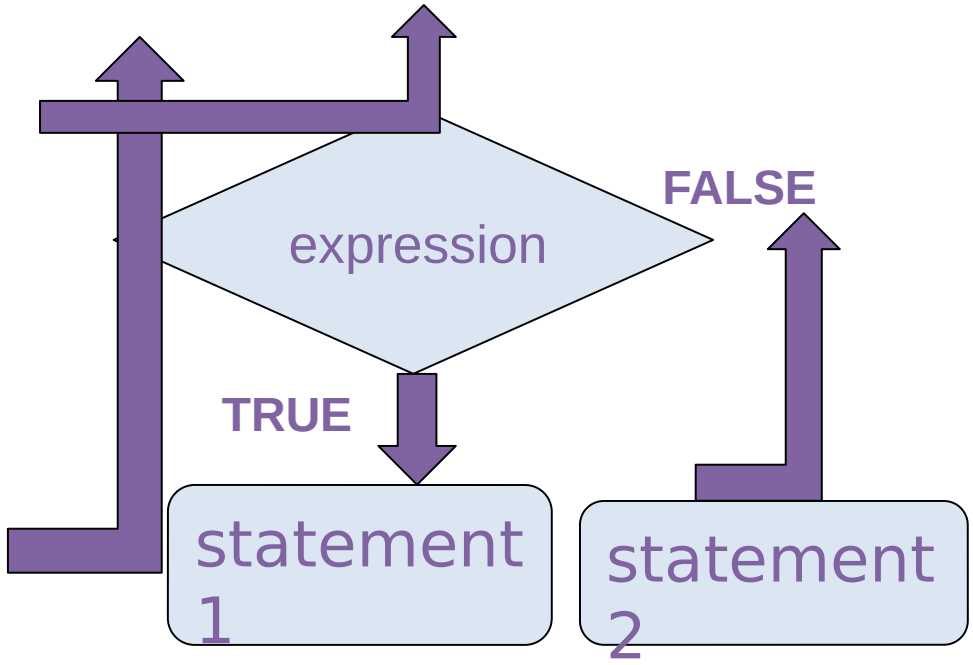

```
scanf("%d", &n);
int i = 1;

while (i <= 10) {
   printf("%d X %d = %d",
              n, i, n*i);
   i = i + 1;
}

// loop exited!
```

While Statement

```
while (expression)
        statement1;
statement2;
```



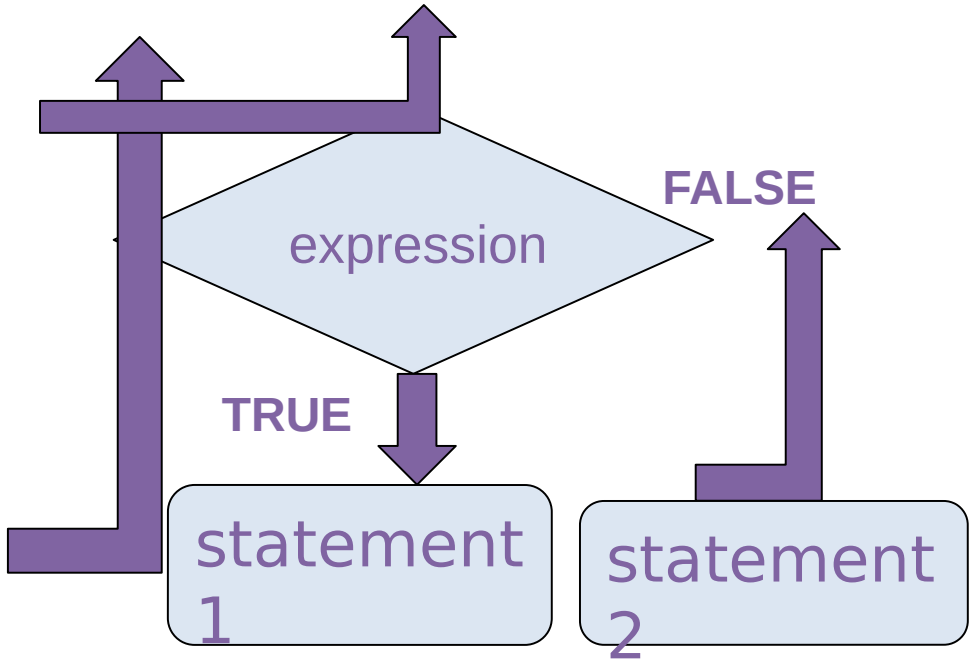
Read in English as:

As long as expression is TRUE execute statement1.

when expression becomes FALSE execute statement 2.

While Statement

```
while (expression)
        statement1;
statement2;
```



1. Evaluate expression
2. If TRUE then
   a) execute statement1
   b) goto step 1.
3. If FALSE then execute statement2.

Example 1

1. Read a sequence of integers from the terminal until -1 is read
2. Output sum of numbers read, not including the -1

First, let us write the loop, then add code for sum

```
int a;
scanf("%d", &a);                /* read into a */
while ( a !=  -1) {
        scanf("%d", &a);    /* read into a
inside loop*/
}
```

Tracing the Loop

```
int a;
scanf("%d", &a);                    /* read into a */
while ( a !=  -1) {
        scanf("%d", &a);  /*read into a inside loop*/
}
```

INPUT

4
15
-5
-1

-1

Trace of memory location a

- *One scanf is executed every time body of the loop is executed.*
- *Every scanf execution reads one integer.*

Add Numbers Until -1

- Keep an integer variable s.
- s is the sum of the numbers seen so far (except the -1).

```
int a;
int s;
s = 0; // not seen any a yet
scanf("%d", &a);      // read into a
while (a !=  -1) {
      s = s + a;       // last a is not -1
      scanf("%d", &a);  // read into a inside
loop
}
```

Terminology

- Iteration: Each run of the loop is called an iteration.
  - In example, the loop runs for 3 iterations, corresponding to inputs 4, 15 and -5.
  - For input -1, the loop is exited, so there is no iteration for input -1.
- 3 components of a while loop
  - Initialization
    - first reading of a in example
  - Condition (evaluates to a Boolean value)
    - a != -1
  - Update
    - another reading of a

```
scanf("%d", &a);      /* read into a */

while (a != -1) {
    s = s + a;
    scanf("%d", &a);  /*read into a inside
loop*/
}
```

Common Mistakes

- Initialization is not done
  - Incorrect results. Might give error.
- Update step is skipped
  - Infinite loop: The loop goes on forever. Never terminates.
  - The update step must take the program towards the condition evaluating to false.
- Incorrect termination condition
  - Early or Late exit (even infinite loop).

Practice Problem

- Given a positive integer n, print all the integers less than or equal to n that are divisible by 3 or divisible by 5
- Hint: Two conditions will be used:
  - x <= n
  - (x%3 == 0) || (x%5 == 0)

Program

```
int main ( )
{
    int n; int x;
    scanf(“%d”, &n);      // input n

    x = 1;                    // [while] initialization
    while ( x <= n) {     // [while] condition
        if ((x%3 == 0) || (x%5 == 0)) { // [if] condition
            printf(“%d\n”, x);
        }
        x = x+1;             // [while] update
    }
    return  0;
}
```

do-while Loops

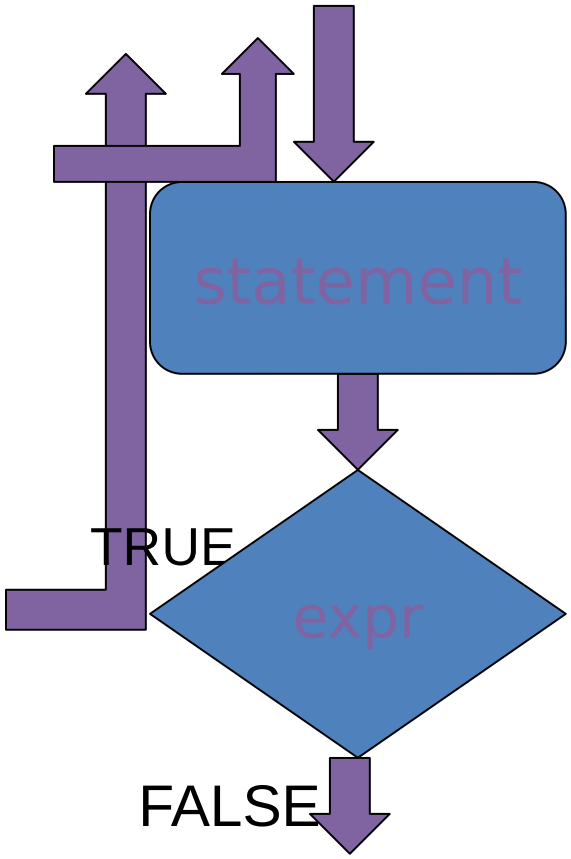
- do-while statement is a variant of while.

General form:

```
do

statement
while (expr);
```

- Execution:
  1. First execute statement.
  2. Then evaluate expr.
  3. If expr is TRUE then go to step 1.
  4. If expr is FALSE then break from loop



- Continuation of loop is tested after the statement.

Comparing while and do-while

- In a while loop the body of the loop may not get executed even once, whereas, in a do-while loop the body of the loop gets executed at least once.
- In the do-while loop structure, there is a semicolon after the condition of the loop.
- Rest is similar to a while loop.

Comparative Example

- Problem: Read integers and output each integer until -1 is seen (include -1 in output).
- The program fragments using while and do-while.

Using do-while

```
int a; /*current int*/

do {
        scanf("%d", &a);
        printf("%d\n", a);
} while (a != -1);
```

Using while

```
int a;/*current int*/

scanf("%d",&a);
while (a != -1) {
        printf("%d\n",
a);
        scanf("%d", &a);
}
printf("%d\n", a);
```

For loop in C

- General form

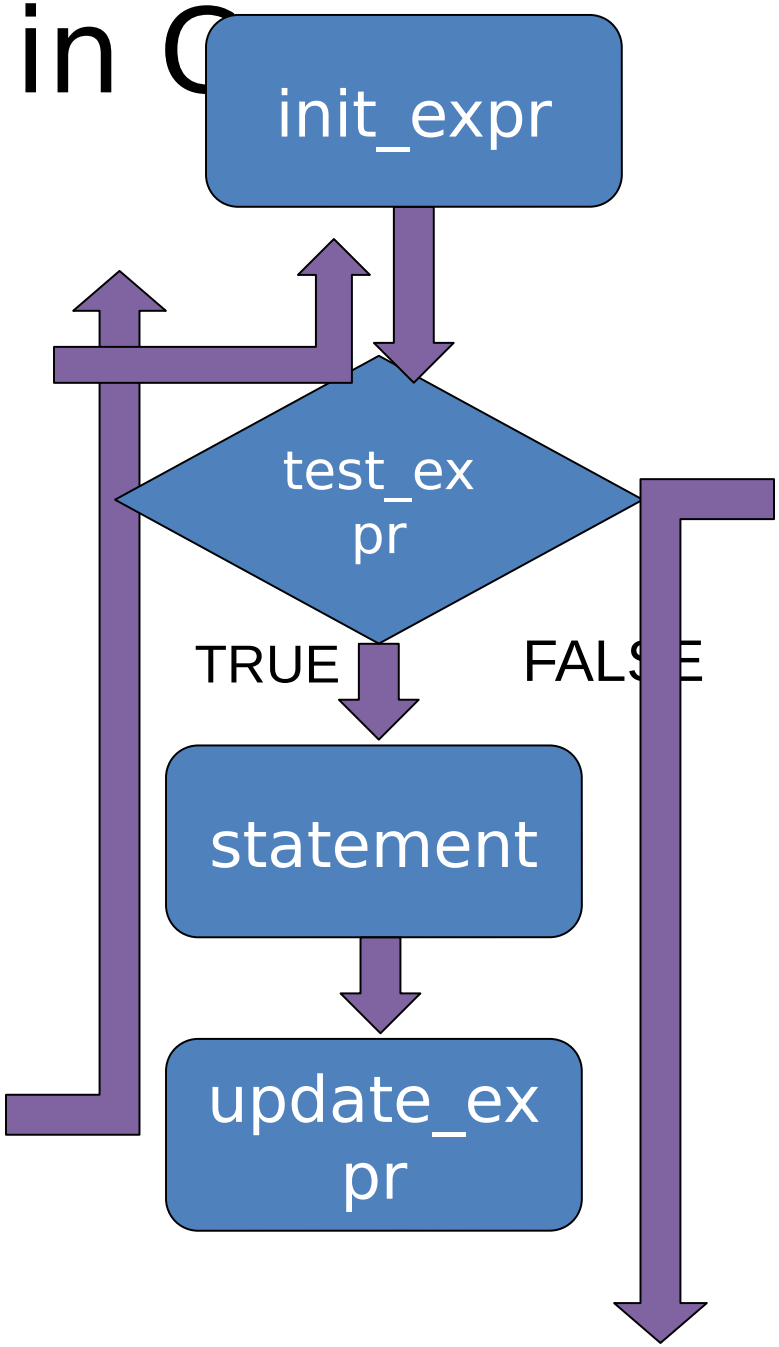
```
for (init_expr; test_expr;
update_expr)
    statement;
```

- init_expr is the initialization expression.
- update_expr is the update expression.
- test_expr is the expression that evaluates to either TRUE (non-zero) or FALSE (zero).
- statement is the work to repeat (can be multiple statements in {…})

For Loop in C

```
for (init_expr; test_expr;
update_expr)
        statement;
```

1. *First evaluate init_expr;*
2. *Evaluate test_expr;*
3. *If test_expr is TRUE then*
   a) *execute statement;*
   b) *execute update_expr;*
   c) *go to Step 2.*
4. *if test_expr is FALSE then break from the loop*



For Loop

- *Print the sum of the reciprocals of the first 100 natural numbers.*

```
int i; // counter from 1..100
float rsum = 0.0;// the sum

// the for loop
for ( i=1; i<=100; i=i+1 ) {
    rsum = rsum +  (1.0/i);
}
printf("sum is %f ", rsum);
```

```
int i;
float rsum = 0.0;

for (i=1; i<=4; i=i+1) {
rsum = rsum + (1.0/i);
}

printf("sum is %f", rsum);
```

i	rsum
5	2.0833333..

1. Evaluate init_expr; i.e., i=1;
2. Evaluate test_expr i.e., i<=4 TRUE
3. Enter body of loop and execute.
4. Execute update_expr; i=i+1; i is 2
5. Evaluate test_expr i<=4: TRUE
6. Enter body of loop and execute.
7. Execute i=i+1; i is 3
8. Evaluate test_expr i<=4: TRUE
9. Enter body of loop and execute.
10. Execute i=i+1; i is 4
11. Evaluate test_expr i<=4: TRUE
12. Enter body of loop and execute.
13. Execute i=i+1; i is 5
14. Evaluate test_expr i<=4: FALSE
15. Exit loop & jump to printf

sum is 2.083333

sum of reciprocals of 1 to 4 is 2.08333

for Loop in Terms of while Loop

```
for (init_expr; test_expr; update_expr)
    statement;
```

- Execution is (almost) equivalent to

```
init_expr;
while (test_expr)
{
    statement;
    update_expr;
}
```

- Almost? Exception if there is a continue; inside statement– this will be covered later.
- Both are (almost) equivalent in power.
- Which loop structure to use, depends on the convenience of the programmer.

Example: Geometric Progression

- Given positive real numbers r and a, and a positive integer, n, the nth term of the geometric progression with a as the first term and r as the common ratio is ar^{n-1}.
- Write a program that given r, a, and n, displays the first n terms of the corresponding geometric progression.

Program: Geometric Progression

```
#include <stdio.h>

int main(void) {
    int n, i;
    float r, a, term;

    // Reading inputs from the user
    scanf(“%f”, &r);
    scanf(“%f”, &a);
    scanf(“%d”, &n);
    term = a;
    for (i = 1; i <= n; i++)
    {
        printf(%f\n, term);    // Displaying the i-th term
        term = term * r;        // Computing the (i+1)-th term
    }
    return 0;
}
```

Program: Geometric Progression

```
#include <stdio.h>

int main(void) {
    int n, i;
    float r, a, term;

    // Reading inputs from th
    scanf("%f", &r);
    scanf("%f", &a);
    scanf("%d", &n);

    term = a;
    for (i = 1; i <= n; i++)
    {
        term = term * r;        // Computing the (i+1)-th term
        printf(%f\n, term);     // Displaying the (i+1)-th term
    }
    return 0;
```

Careful: Changing the order of the statements will change the meaning of the program.
a ar ar^2 ar^3 ....
Vs
ar ar^2 ar^3 ar^4
....

Next Class

- Using loops
  - Use cases
  - Warnings and exception handling